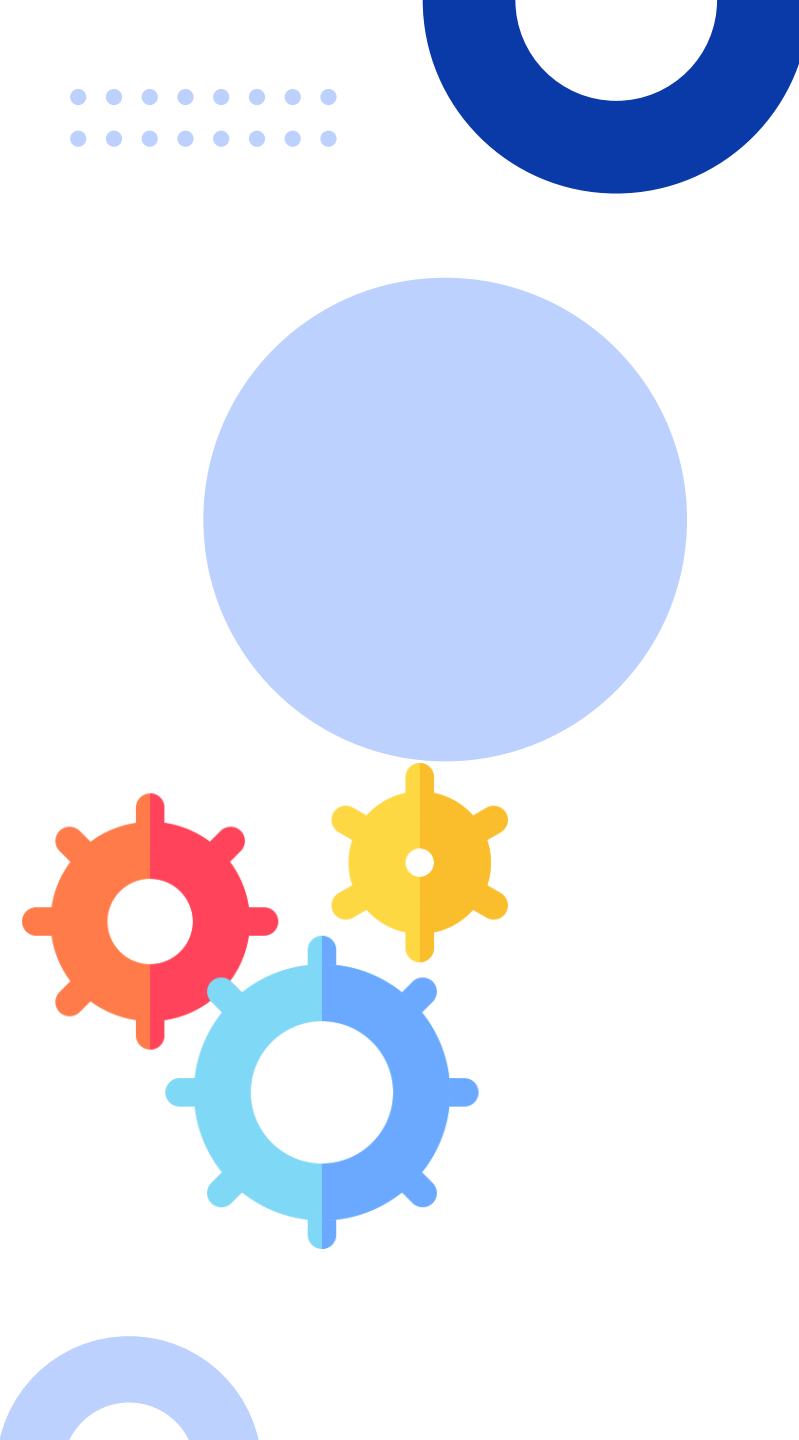


บทที่ 7 ฟังก์ชัน

Function

อาจารย์ผู้สอน:

- ▶ อ. ดร.พงษ์ศร จันทรียอย
วิทยาลัยการคอมพิวเตอร์ มหาวิทยาลัยขอนแก่น

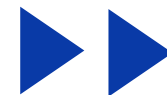


วัตถุประสงค์การเรียนรู้

- ผู้เรียนสามารถอธิบายแนวคิด ความสำคัญ และประโยชน์ของการใช้ฟังก์ชันในการเขียนโปรแกรม
- ผู้เรียนสามารถสร้างฟังก์ชัน เข้าใจส่วนประกอบต่างๆ ของฟังก์ชัน และสามารถเรียกใช้งานฟังก์ชันได้อย่างถูกต้อง
- ผู้เรียนสามารถสร้างฟังก์ชันที่มีการรับพารามิเตอร์ในรูปแบบต่างๆ และส่งค่ากลับมายังโปรแกรมหลักได้
- ผู้เรียนสามารถอธิบายเกี่ยวกับขอบเขตของตัวแปรได้

เนื้อหา

01. ฟังก์ชัน และความสำคัญของฟังก์ชัน
02. การประกาศฟังก์ชัน
03. ฟังก์ชันแบบ Lambda



ความสำคัญของฟังก์ชัน

- จงเขียนโปรแกรมหาผลรวมของตัวเลขที่เก็บในลิสต์

```
a = [1, 2, 3, 4, 5]
b = [-1, -2, -3, -4, -5]
```

```
sum_a = 0
sum_b = 0
```

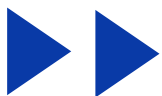
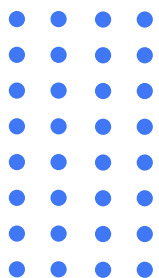
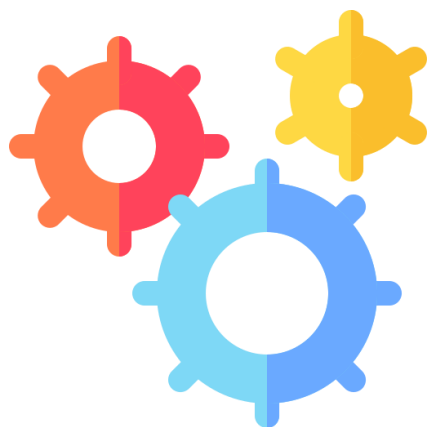
```
for x in a:
    sum_a += x
```

```
for x in b:
    sum_b += x
```

```
print(sum_a)
print(sum_b)
```

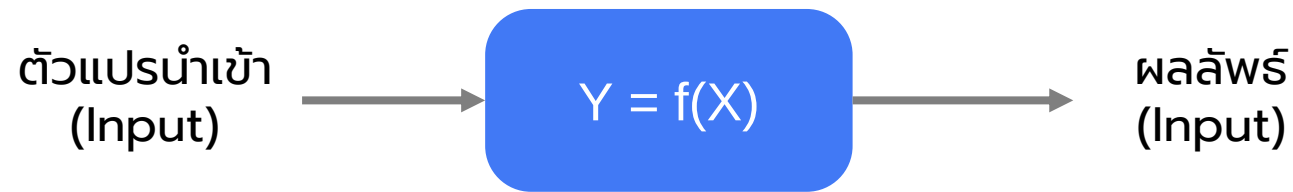
หากมีข้อมูล 1,000 ลิสต์
จะเขียนโค้ดแบบนี้ 1,000 ครั้ง

ฟังก์ชัน
คืออะไร?



ฟังก์ชันในทางคณิตศาสตร์

- ฟังก์ชัน คือ กระบวนการทางคณิตศาสตร์ที่กำหนดความสัมพันธ์แบบแน่นอนระหว่างตัวแปรหนึ่งกับอีกตัวแปรหนึ่งหรือหลายตัวแปร



ฟังก์ชันตรีโกณมิติ

- $\sin(x)$
- $\cos(x)$
- $\tan(x)$

ฟังก์ชันเอ็กซ์โปเนนเชียล และลอการิทึม

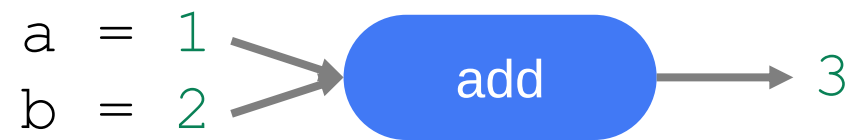
- $\exp(x)$
- e^x
- $\ln(x)$

ฟังก์ชันอื่นๆ

- $\sqrt{x}$
- $x!$

ฟังก์ชันในการเขียนโปรแกรม

- ส่งค่าไป
- ประมวลผล
- คืนค่ากลับมา

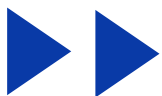
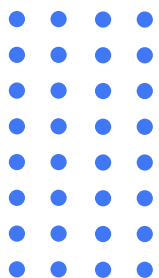
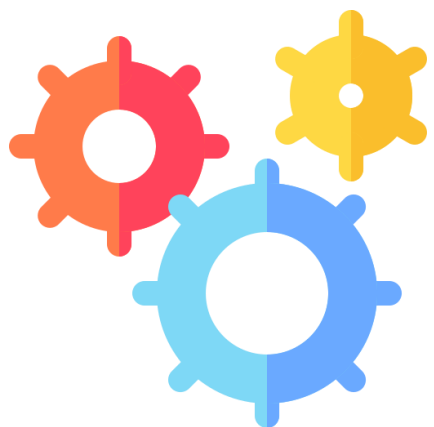


- ใช้งานได้หลายครั้ง

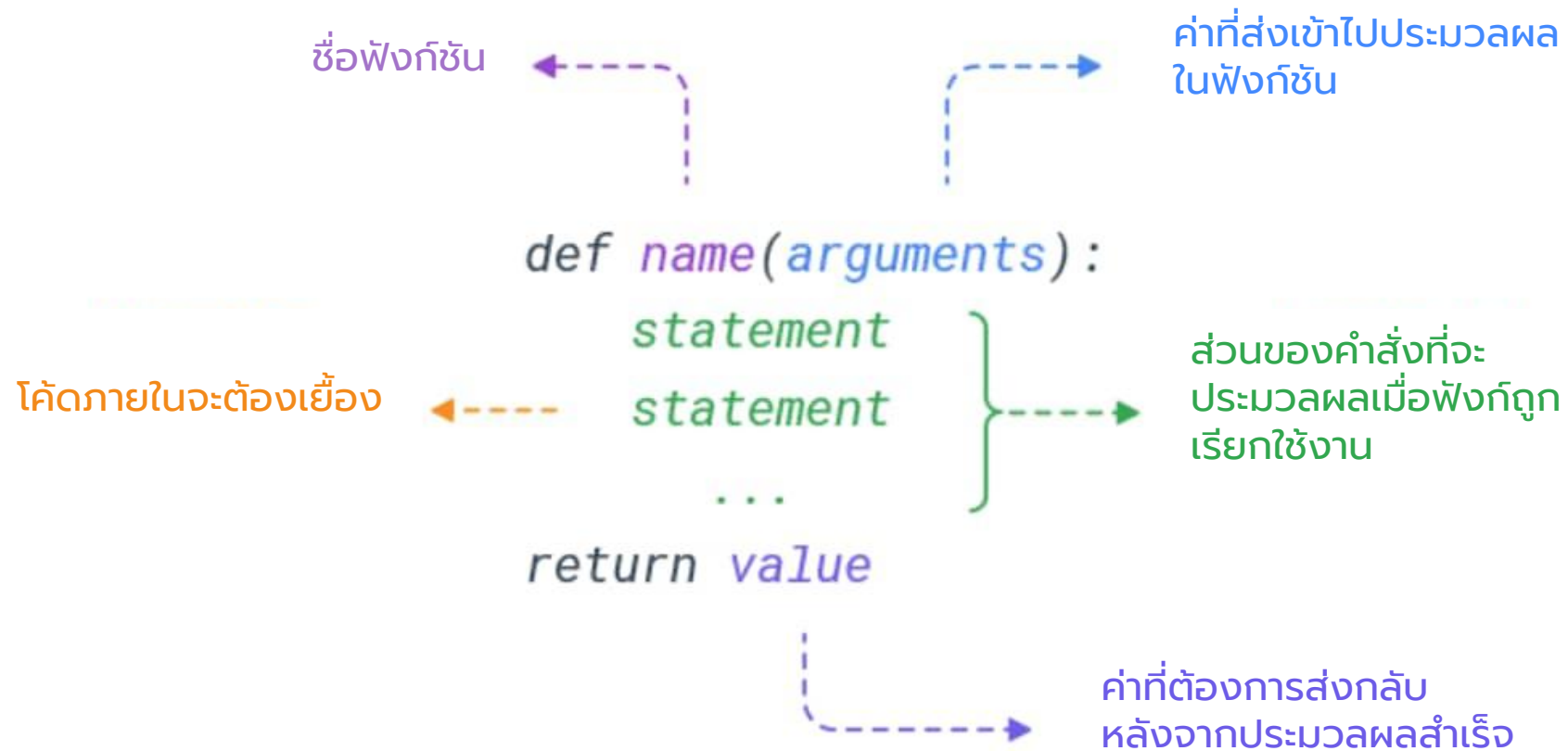
ประเภทของฟังก์ชัน

- Built-in function :
 - คือฟังก์ชันที่**ติดมากับ Python ตั้งแต่ติดตั้ง**
 - ตัวอย่าง Built-in ก็คือฟังก์ชันต่างๆ ที่เราเคยใช้มาในบทก่อนๆ เช่น range(), print(), input(), type(), id(), eval() เป็นต้น.
- User-defined function :
 - เป็นฟังก์ชันที่โปรแกรมเมอร์สร้างขึ้นมา
 - ฟังก์ชันที่คนสร้างขึ้นมาเพื่อทำงานตามวัตถุประสงค์ เช่น หาพื้นที่วงกลม คำนวณ BMI คำนวณดอกเบี๋ย

การประกาศ ฟังก์ชัน



การประกาศฟังก์ชัน



การประกาศฟังก์ชัน

Defining the function:

```
def triple_number(num):  
    tripled = num * 3  
    print(tripled)
```

function name

parameter

colon

indents that designate code inside a function

code word that defines this as a function

การประกาศฟังก์ชัน

- `add()` : ฟังก์ชันบวกเลข 2 จำนวน

```
def add(num1,num2):           # Function name: 'add', Parameters: 'num1', 'num2'
    print("Number 1: ", num1)  # Function body
    print("Number 2: ", num2)  # Function body
    addition = num1 + num2     # Function body
    return addition           # return value
```

เรียกโดยใช้ ชื่อฟังก์ชัน และส่งค่าไปตามที่
กำหนดไว้ตอนสร้าง

```
res1 = add(2, 4)  # เรียกใช้ฟังก์ชันครั้งที่ 1
res2 = add(5, 8)  # เรียกใช้ฟังก์ชันครั้งที่ 2
print("Result: ", res1)
print("Result: ", res2)
```

ฟังก์ชันที่ไม่มีการส่งพารามิเตอร์ (parameters)

- การประกาศฟังก์ชันที่ไม่มีพารามิเตอร์

```
def greet():  
    print("Welcome to Python")
```

```
greet() # เรียกใช้ฟังก์ชัน  
greet() # เรียกใช้ฟังก์ชัน  
greet() # เรียกใช้ฟังก์ชัน
```

> Output

```
Welcome to Python  
Welcome to Python  
Welcome to Python
```

ฟังก์ชันที่ไม่มีการส่งพารามิเตอร์ (parameters)

- การประกาศฟังก์ชันที่ไม่มีพารามิเตอร์

```
def generate_full_name():  
    first_name = 'Pongsathon'  
    last_name = 'Janyoi'  
    space = ' '  
    full_name = first_name + space + last_name  
    print(full_name)  
generate_full_name() # calling a function
```

> Output

Pongsathon Janyoi

ฟังก์ชันแบบมีการรับพารามิเตอร์

- ตัวอย่าง ฟังก์ชันแปลงอุณหภูมิจากฟาเรนไฮต์ (°F) เป็นเซลเซียส (°C)

```
def fahrenheitToCelsius(fahrenheit):  
    celsius = (fahrenheit - 32) * 5 / 9  
    print("Temperature in Celsius: %.2f" %celsius)
```

#ตัวอย่างการเรียกใช้งาน

```
fahrenheit_temp = 68  
fahrenheitToCelsius(fahrenheit_temp)
```

Output

Temperature in Celsius: 20.00

ฟังก์ชันแบบมีการรับพารามิเตอร์

- ตัวอย่าง ฟังก์ชันคำนวณพื้นที่วงกลม

ค่าที่ส่งไปจะถูกตั้งชื่อใหม่เป็น radius

```
def circleArea(radius):
    area = 3.141592653589793 * (radius ** 2)
    print("The area of the circle with radius",radius, "is:", area)
r= 5
circleArea(r)
```

ส่งตัวแปร r ไปประมวลผล

Output

The area of the circle with radius
5 is: 78.53981633974483

ฟังก์ชันแบบมีการรับพารามิเตอร์

- ตัวอย่าง ฟังก์ชันที่มีมากกว่า 1 พารามิเตอร์

```
def addTwoNumbers(num1,num2):  
    total = num1 + num2  
    print('Result:',total)
```

```
addTwoNumbers(5,10)  
addTwoNumbers(11,5)
```

ใช้ "," คั่นหากมีมากกว่า 1
พารามิเตอร์

Output

Result: 15
Result: 16

ฟังก์ชันแบบมีการรับพารามิเตอร์

- การส่งตัวแปร List เข้าไปในฟังก์ชัน

```
def list_sum(lst):  
    total = 0  
    for num in lst:  
        total += num  
  
    print("Sum of list elements:", total)
```

```
my_list = [1, 2, 3, 4, 5]  
list_sum(my_list)
```

> Output

Sum of list elements: 15

ยังไม่สามารถนำผลลัพธ์ที่คำนวณใน
ฟังก์ชันไปใช้งานต่อได้

ฟังก์ชันแบบมีการส่งค่ากลับ

- สามารถใช้คำสั่ง return ตามด้วยค่าหรือตัวแปรที่ต้องการส่งกลับ
- หากไม่มีการ return จากมีค่าเป็น None
- ตัวอย่าง ฟังก์ชัน PI() ซึ่งส่งกลับค่า 3.141592653589793 เพื่อใช้คำนวณเส้นรอบวงหรือพื้นที่ของวงกลม

```
def PI():  
    return 3.141592653589793
```

```
r = 10  
print(2*PI()*r)
```

> Output

62.83185307179586

ฟังก์ชันแบบมีการส่งค่ากลับ

- ตัวอย่าง กรณีส่งค่ากลับเป็นข้อความ

```
def greet():  
    return "Hello "  
  
print(greet()+" Somchai")  
print(greet()+" Somphong")
```

 Output

```
Hello Somchai  
Hello Somphong
```

ฟังก์ชันแบบมีการส่งค่ากลับ

- กรณีที่ฟังก์ชันมีพารามิเตอร์มากกว่าหนึ่งตัว ให้ใช้เครื่องหมาย “,” คั่น

```
def addTwoNumbers(num1,num2):  
    total = num1 + num2  
    return total
```

#การเรียกใช้งาน

```
x1,y1 = 5,10
```

```
x2,y2 = 10,5
```

```
result1 = addTwoNumbers(x1,y1)
```

```
result2 = addTwoNumbers(x2,y2)
```

```
print(f'{x1} + {y1} = {result1}')
```

```
print(f'{x2} + {y2} = {result2}')
```

 Output

```
5 + 10 = 15
```

```
10 + 5 = 15
```

ฟังก์ชันแบบมีการส่งค่ากลับ

- ตัวอย่าง ฟังก์ชัน `list_sum(lst)` รับพารามิเตอร์ที่มีชนิดข้อมูลเป็น `list` หาผลรวมของสมาชิกทุกตัว

```
def list_sum(lst):  
    total = 0  
    for num in lst:  
        total += num  
    return total
```

```
my_list = [1, 2, 3, 4, 5]  
result = list_sum(my_list)  
print("Sum of list elements:", result)
```

 Output

Sum of list elements: 15

ฟังก์ชันแบบมีการส่งค่ากลับ

- หากมีการส่งค่ากลับมากกว่า 1 ค่า สามารถใช้เครื่องหมาย “,” คั่น

```
def arithmetic(num1, num2):
    add = num1 + num2
    sub = num1 - num2
    multiply = num1 * num2
    division = num1 / num2
    # return four values
    return add, sub, multiply, division
```

ส่งกลับมาเป็น tuple 4 ค่า

```
a, b, c, d = arithmetic(10, 2)
```

```
print("Addition: ", a)
print("Subtraction: ", b)
print("Multiplication: ", c)
print("Division: ", d)
```

```
r = arithmetic(50, 2)
print("\nAddition: ", r[0])
print("Subtraction: ", r[1])
print("Multiplication: ", r[2])
print("Division: ", r[3])
```

 Output

```
Addition: 12
Subtraction: 8
Multiplication: 20
Division: 5.0
```



Question 1



Question 2-A

A decorative vertical line of 16 dots, alternating between light pink and dark red colors, located on the left side of the slide.

Question 2-B



Question 3

การกำหนดรูปแบบการส่งพารามิเตอร์

- Required Arguments คือ พารามิเตอร์ที่ต้องมีการกำหนดค่าเสมอ
- การเรียกใช้งานฟังก์ชันจะต้องมีการส่งพารามิเตอร์ไปให้ครบทั้งหมด

ต้องส่ง Argument 1 ตัว

```
def greeting(name): #พารามิเตอร์ 1 ตัว
    message = name + ', welcome to Python course'
    return message
print(greeting('Many'))
```

ต้องส่ง Argument 2 ตัว

```
def calculate_age(current_year, birth_year):
    age = current_year - birth_year
    return age
print('Age: ', calculate_age(2021, 1989)) #35
```

การกำหนดรูปแบบการส่งพารามิเตอร์

- ตัวอย่าง การส่งพารามิเตอร์ไม่ครบ

```
def calculate_age(current_year, birth_year):  
    age = current_year - birth_year  
    return age
```

ต้องส่ง Argument 2 ตัว

```
print('Age: ', calculate_age(2024))
```

Traceback (most recent call last):

File "[D:\python\hello.py](#)", line 747, in <module>

```
print('Age: ', calculate_age(2024))
```

^^^^^^^^^^^^^^^^^^^^^^^^^^^^

TypeError: calculate_age() missing 1 required positional argument: 'birth_year'

การส่งพารามิเตอร์ด้วยการระบุชื่อของพารามิเตอร์

- สามารถส่งพารามิเตอร์ในลักษณะที่ระบุชื่อได้ และไม่จำเป็นต้องเรียงลำดับตามที่กำหนดไว้ในฟังก์ชัน

```
def calculate_age (current_year, birth_year):
    age = current_year - birth_year
    return age
```

#แบบไม่ระบุ key ต้องเรียกตามลำดับที่กำหนดไว้

```
print('Age: ', calculate_age(2023, 1989))
```

#แบบระบุ key=value ส่ง birth_year เป็นตัวแรกก็สามารถทำได้

```
print('Age: ', calculate_age(birth_year=1989, current_year=2023))
```

Output

Age: 34

Age: 34

ชื่อพารามิเตอร์ = ค่า

การกำหนดค่าเริ่มต้นของพารามิเตอร์ (Default arguments)

- ภาษา Python สามารถที่จะกำหนดค่าเริ่มต้นของพารามิเตอร์บางตัวได้
- มีประโยชน์ในกรณีที่มีพารามิเตอร์จำนวนมาก แต่**บางพารามิเตอร์เป็นพารามิเตอร์เสริมเท่านั้น**

```
def function_name(required_param1, param2=default_value2, param3=default_value3):  
    return param1+ param3  
  
r1 = function_name(val1, param2=val2)  
r2 = function_name(val1, param3=val3)
```

- พารามิเตอร์หลัก จะต้องอยู่ก่อนพารามิเตอร์ที่กำหนดค่าเริ่มต้นเสมอ
- สามารถกำหนดค่าเริ่มต้นของพารามิเตอร์ทุกตัวได้

การกำหนดค่าเริ่มต้นของพารามิเตอร์ (Default arguments)

- ตัวอย่าง การกำหนดค่าเริ่มต้นพารามิเตอร์

```
def greet(name, greeting="Hello") :  
    print(f"{greeting}, {name}!")
```

#ส่งค่าทุกพารามิเตอร์

```
greet("Man", "Hi")
```

ส่งค่าพารามิเตอร์ 'name' ตัวเดียว

```
greet("Python")
```

ไม่ส่งค่าไป greeting จะถูก
กำหนดเป็น Hello

Output

```
Hi, Man!  
Hello, Python!
```

การกำหนดค่าเริ่มต้นของพารามิเตอร์ (Default arguments)

- ตัวอย่าง ฟังก์ชัน `calculate_age()` สำหรับคำนวณอายุจากปีเกิด

```
def calculate_age (b_year, cur_year=2023):  
    age = cur_year - b_year  
    return age  
  
print('Age: ', calculate_age(1989))
```

ไม่มีการส่ง `cur_year` ไปจะถูก
กำหนดค่าเป็น 2023

Output

Age: 34

การกำหนดค่าเริ่มต้นของพารามิเตอร์ (Default arguments)

- ตัวอย่าง ฟังก์ชัน `time_to_minutes()` ที่มีการกำหนดค่าเริ่มต้นของพารามิเตอร์ทุกตัว

```
def time_to_minutes(days=0, hours=0, minutes=0):
    total_minutes = days * 24 * 60 + hours * 60 + minutes
    return total_minutes
```

ทุกพารามิเตอร์มีค่าเริ่มต้น

```
days = 2; hours = 5; minutes = 30
```

```
total_minutes = time_to_minutes(days, hours, minutes)
print("Total minutes:", total_minutes)
```

```
total_minutes = time_to_minutes(hours=hours)
print("Total minutes:", total_minutes)
```

ส่งเฉพาะ hours ไป

Output

```
Total minutes: 3210
Total minutes: 300
```



Question 4

พารามิเตอร์แบบไม่กำหนดจำนวน (Variable-length Arguments)

- ฟังก์ชันยังสามารถกำหนดพารามิเตอร์แบบที่ไม่กำหนดจำนวนหรือระบุว่าพารามิเตอร์ที่ตัวนั่นเอง
- มีประโยชน์ในลักษณะที่**ไม่ทราบจำนวนพารามิเตอร์แน่ชัด**

```
def function_name(*args):  
    # Function body  
# ตัวอย่างการใช้งาน  
function_name(arg1, arg2, arg3, ...)
```

ตัวอย่าง build-in function :

```
print('a', 'b', 'c')
```

ภายในฟังก์ชันจะมองเห็นพารามิเตอร์ args เป็นตัวแปรชนิด tuple ซึ่งจะต้องนำค่าไปใช้งานในลักษณะ tuple

พารามิเตอร์แบบไม่กำหนดจำนวน (Variable-length Arguments)

- ภายในฟังก์ชันจะมองเห็นพารามิเตอร์เป็นตัวแปรชนิด tuple
- ตัวอย่าง **ฟังก์ชัน greet()** โดยรับพารามิเตอร์แบบไม่กำหนดจำนวน

```
def greet(*names):
    print(type(names))
    for name in names:
        print("Hello", name)
```

names จะเป็น tuple

ส่ง 4 ตัว

```
greet("Python", "Java", "C", "C++")
greet("Numpy", "Scikit-learn")
```

ส่ง 2 ตัว

Output

```
<class 'tuple'>
Hello Python
Hello Java
Hello C
Hello C++
<class 'tuple'>
Hello Numpy
Hello Scikit-learn
```

พารามิเตอร์แบบไม่กำหนดจำนวน (Variable-length Arguments)

- ฟังก์ชัน `sum_numbers(*args)` โดยรับพารามิเตอร์แบบไม่กำหนดจำนวน

```
def sum_numbers(*args):  
    print(type(args))  
    total = 0  
    for num in args:  
        total += num  
    return total  
  
result1 = sum_numbers(1, 2, 3, 4, 5)  
print(result1)    # Output: 15  
  
result2 = sum_numbers(10, 20, 30)  
print(result2)    # Output: 60
```

> Output

```
<class 'tuple'>  
15  
<class 'tuple'>  
60
```



Question 5

ตัวอย่างฟังก์ชัน



- ตัวอย่าง: ฟังก์ชันประเมินราคารถยนต์มือสอง

- age (อายุรถ)
- mileage (ระยะทางที่วิ่ง)
- accident (ประวัติการชน)

```
def estimate_car_price(age, mileage, accident):
    if age <= 5:
        if mileage <= 50000:
            if accident == 0:
                return 500000
            else:
                return 450000
        else:
            return 400000
    else:
        if mileage <= 100000:
            return 300000
        else:
            return 200000
```

Output

```
Enter car age (years): 3
Enter mileage (km): 40000
Accident? (1=yes, 0=no): 0
Estimated price: 500000 THB
```

```
age = int(input("Enter car age (years): "))
mileage = int(input("Enter mileage (km): "))
accident = int(input("Accident? (1=yes, 0=no): "))

price = estimate_car_price(age, mileage, accident)
print("Estimated price:", price, "THB")
```

ตัวอย่างฟังก์ชัน

- ตัวอย่าง: ฟังก์ชันตรวจสอบระดับน้ำตาลในเลือด

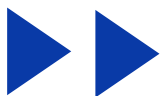
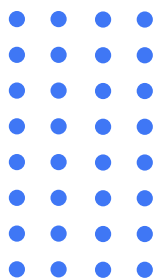
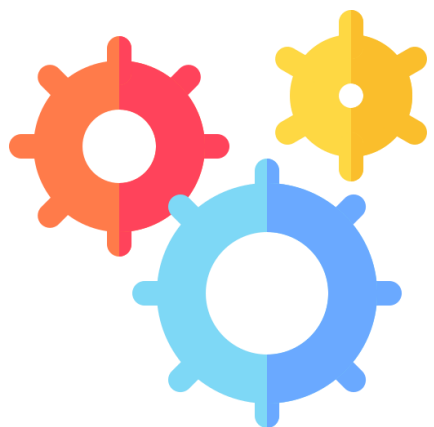
```
def check_blood_sugar(blood_sugar):  
    if blood_sugar < 100:  
        return "Normal"  
    elif blood_sugar < 126:  
        return "Prediabetes"  
    else:  
        return "Diabetes"
```

```
n = int(input("Enter number of patients:"))  
for i in range(n):  
    value = float(input(f"Enter blood sugar level #{i+1} (mg/dL): "))  
    result = check_blood_sugar(value)  
    print("Result:", result)
```

>- Output

```
Enter number of patients:3  
Enter blood sugar level #1 (mg/dL): 90  
Result: Normal  
Enter blood sugar level #2 (mg/dL): 110  
Result: Prediabetes  
Enter blood sugar level #3 (mg/dL): 150  
Result: Diabetes
```

ฟังก์ชันแบบ Lambda



ฟังก์ชันแบบ Lambda

- Lambda expression เป็นรูปแบบสั้นๆ ของการสร้างฟังก์ชันแบบไม่ต้องระบุชื่อในภาษา Python
- ใช้เพื่อสร้างฟังก์ชันแบบอินไลน์ (inline) ที่สามารถใช้งานได้โดยตรงในที่ที่ต้องการ
- มักจะใช้ในกรณีที่ต้องการฟังก์ชันที่ไม่ซับซ้อน และใช้งานเพียงครั้งเดียวหรือในบริบทที่จำกัด
- ใช้ keyword lambda แทน def

```
var = lambda arguments: expression
```

```
var = lambda [arg1, [arg2, .. argn]]: expression
```

```
print("value from lambda:", var(arg1, agr2, .. argn))
```

ฟังก์ชันแบบ Lambda

- ตัวอย่าง ฟังก์ชัน lambda คำนวณผลบวกของ 2 จำนวน

สร้าง Lambda expression

add = lambda a, b: a + b

print(add(5, 3))

รับพารามิเตอร์ a และ b

คืนค่า a + b

> Output

8

ฟังก์ชันแบบ Lambda

- ตัวอย่าง ฟังก์ชัน lambda คำนวณผลยกกำลังของ 2 จำนวน

สร้าง Lambda expression

power = lambda a, b: a ** b

print(power(2, 3))

รับพารามิเตอร์ a และ b

คืนค่า a + b

> Output

8

การใช้ฟังก์ชัน lambda ร่วมกันฟังก์ชัน map()

- `map(func,iter)` : เป็นฟังก์ชันสำหรับเอาข้อมูลที่บรรจุในตัวแปรเป็นชนิดพวก `list, tuple` ไปผ่านฟังก์ชันทีละบุ

```
numbers = [1, 2, 3, 4, 5]
```

```
def square(x):  
    return x ** 2
```

```
squared_numbers = list(map(square, numbers))  
print(squared_numbers)
```

ชื่อฟังก์ชัน

ส่ง numbers ไป

> Output

[1, 4, 9, 16, 25]

การใช้ฟังก์ชัน lambda ร่วมกันฟังก์ชัน map()

- `map(func,iter)` : เป็นฟังก์ชันสำหรับนำข้อมูลที่บรรจุในตัวแปรเป็นชนิด list, tuple ไปผ่านฟังก์ชันทีละบุ

```
numbers = [1, 2, 3, 4, 5]
```

```
def square(x):  
    return x ** 2
```

```
squared_numbers = list(map(square, numbers))  
print(squared_numbers)
```

ชื่อฟังก์ชัน

ส่ง numbers ไป

> Output

[1, 4, 9, 16, 25]

การใช้ฟังก์ชัน lambda ร่วมกันฟังก์ชัน filter()

- filter() : เป็นฟังก์ชันสำหรับกรองเฉพาะค่าที่ทดสอบกับฟังก์ชันแล้วผลลัพธ์เป็น True

```
def is_even(num):  
    return num % 2 == 0
```

คืน True หรือ False

```
numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]  
filtered_numbers = list(filter(is_even, numbers))  
  
print(filtered_numbers)
```

กรองเอาเฉพาะที่
ผลลัพธ์เป็น True

> Output

[2, 4, 6, 8, 10]

การใช้ฟังก์ชัน lambda ร่วมกันฟังก์ชัน filter()

- `filter()` : เป็นฟังก์ชันสำหรับกรองเฉพาะค่าที่ทดสอบกับฟังก์ชันแล้วผลลัพธ์เป็น `True`

```
numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
filtered_numbers = list(filter(lambda x: x % 2 == 0, numbers))
```

> Output

[2, 4, 6, 8, 10]



Question 6

Question 7

- หากรันโค้ดต่อไปนี้จะได้ผลลัพธ์อย่างไร

ตัวอย่างฟังก์ชัน

- ตัวอย่าง: การจัดการข้อมูลและกรองข้อมูลด้วย map() และ filter()

```
data = [
    [3.5, 28, "cat"],
    [4.2, 30, "cat"],
    [12.0, 35, "dog"],
    [15.0, 40, "dog"],
    [0.5, 15, "cat"]]
```

ข้อมูลดิบ: [weight (kg),
tail_length (cm), label]

ข้อมูลผิดปกติ
(outlier)

ใช้ filter() เพื่อลบข้อมูลที่
ผิดปกติ (น้ำหนักน้อยเกินไป)

```
filtered_data = list(filter(lambda x: x[0] > 1.0, data))
```

```
processed_data = list(map(lambda x: [x[0], x[1]/100, x[2]],
    filtered_data))
```

```
print("Raw data: ", data)
print("Filtered Data: ", filtered_data)
print("Processed Data:", processed_data)
```

ใช้ map() เพื่อแปลงหน่วย
(cm → m)

ตัวอย่างฟังก์ชัน

- ตัวอย่าง: การจัดการข้อมูลและกรองข้อมูลด้วย map() และ filter()

```
data = [
    [3.5, 28, "cat"],
    [4.2, 30, "cat"],
    [12.0, 35, "dog"],
    [15.0, 40, "dog"],
    [0.5, 15, "cat"]]
```

ข้อมูลดิบ (input)

ข้อมูลที่ได้ (output)

```
filtered_data = list(filter(lambda x: x[2] in ['cat'], data))
```

```
processed_data = list(map(lambda x: [x[0], x[1]/100, x[2]],
    filtered_data))
```

```
print("Raw data:", data)
print("Filtered Data:", filtered_data)
print("Processed Data:", processed_data)
```

Output

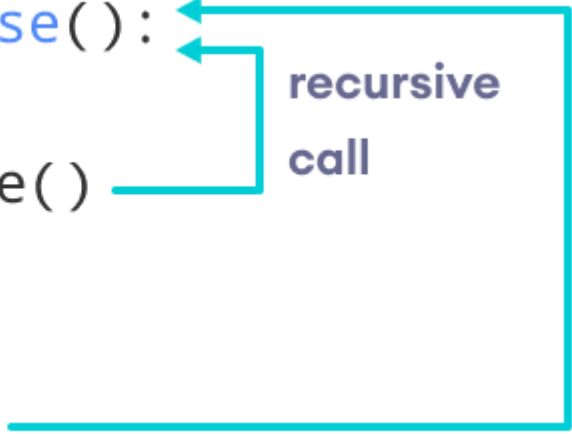
```
Raw data:      [[3.5, 28, 'cat'], [4.2, 30, 'cat'], [12.0, 35, 'dog'],
[15.0, 40, 'dog'], [0.5, 15, 'cat']]
Filtered Data: [[3.5, 28, 'cat'], [4.2, 30, 'cat'], [12.0, 35, 'dog'],
[15.0, 40, 'dog']]
Processed Data: [[3.5, 0.28, 'cat'], [4.2, 0.3, 'cat'], [12.0, 0.35,
'dog'], [15.0, 0.4, 'dog']]
```

ใช้ map() เพื่อแปลงหน่วย
(cm → m)

ฟังก์ชันแบบเรียกตัวเอง (Recursive Function)

- ฟังก์ชันแบบเรียกตัวเอง (Recursive Function) คือ ฟังก์ชันที่มีการเรียกตัวเองภายในฟังก์ชัน

```
def recurse():  
    ...  
    recurse()  
    ...  
  
recurse()
```



recursive call

ฟังก์ชันแบบเรียกตัวเอง (Recursive Function)

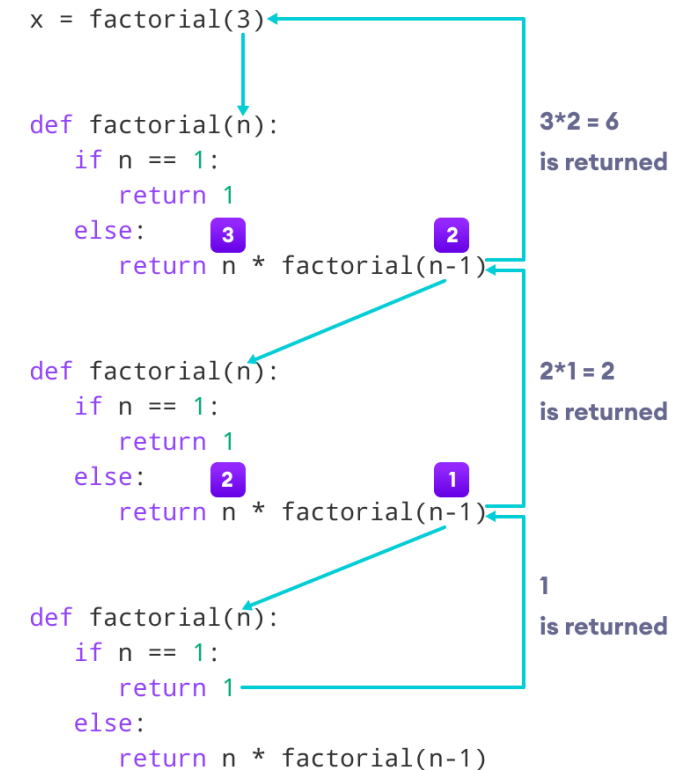
- ตัวอย่าง : factorial

factorial(3)	# 1st call with 3
3 * factorial(2)	# 2nd call with 2
3 * 2 * factorial(1)	# 3rd call with 1
3 * 2 * 1	# return from 3rd call as number=1
3 * 2	# return from 2nd call
6	# return from 1st call

```
def factorial(n):
    if n == 1:
        return 1
    else:
        return (n * factorial(n-1))

num = 3
print(str(num) + "! is", factorial(num))
```

เรียกตัวเอง



ฟังก์ชันแบบเรียกตัวเอง (Recursive Function)

```
def factorial(n):
    if n == 1:
        return 1
    else:
        return (n * factorial(n-1))

num = 10000
print(str(num) + "! is", factorial(num))
```

```
-----
RecursionError                                Traceback (most recent call last)
<ipython-input-7-ad2026247d69> in <cell line: 8>()
      6
      7 num = 10000
----> 8 print(str(num) + "! is", factorial(num))
```

```
----- 1 frames -----
... last 1 frames repeated, from the frame below ...
```

```
<ipython-input-7-ad2026247d69> in factorial(n)
      3     return 1
      4     else:
----> 5     return (n * factorial(n-1))
      6
      7 num = 10000
```

```
RecursionError: maximum recursion depth exceeded in comparison
```

สามารถเรียกตัวเองซ้ำได้ไม่เกิน
1000 ครั้ง

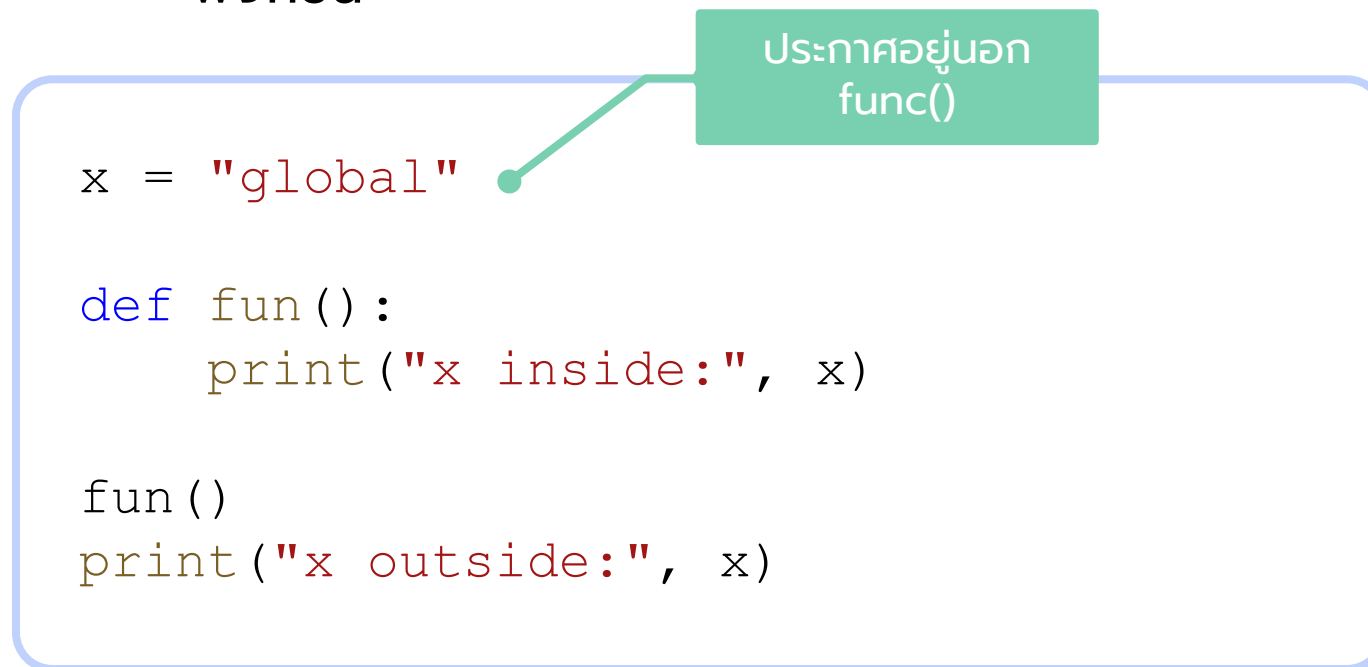
Question 8

- หากรันโค้ดต่อไปนี้จะได้ผลลัพธ์อย่างไร



ตัวแปรแบบ Global และแบบ Local

- **ตัวแปร Global** คือ ตัวแปรที่อยู่นอกฟังก์ชัน
 - ภายในฟังก์ชันจะสามารถเข้าถึงตัวแปร Global ได้ ไม่ว่าจะประกาศตัวแปรไว้ก่อนหรือหลังฟังก์ชัน



```

>_- Output

x inside: global
x outside: global

```

ตัวแปรแบบ Global และแบบ Local

- ตัวแปร Local คือ ตัวแปรที่อยู่ในฟังก์ชัน
 - ภายนอกฟังก์ชันจะสามารถเข้าถึงตัวแปร Local ไม่ได้

```
x = "global"
```

x ประกาศอยู่นอก
fun()

```
def fun():
```

```
    x = 'local'
```

x ประกาศอยู่ใน
fun()

```
    print("x inside:", x, id(x))
```

```
fun()
```

```
print("x outside:", x, id(x))
```

Output

```
x inside: local 134661940500144
x outside: global 134661942569520
```

ตัวแปรแบบ Global และแบบ Local

- ตัวอย่าง การแก้ไขข้อมูลในตัวแปร global ภายในฟังก์ชัน

```
result = 0    #ตัวแปร global

def add_numbers(a, b):
    global result
    result = a + b
    print('Result inside func :',result)

# เรียกใช้งานฟังก์ชัน
add_numbers(5, 3)
# แสดงผลลัพธ์
print('Result outside func : ',result)
```

> Output

Result inside func : 8
Result outside func : 8

ทบทวนท้ายบท

- ฟังก์ชัน คือ ชุดคำสั่งที่ถูกรวมไว้เพื่อทำงานเฉพาะอย่าง
 - ช่วยลดความซ้ำซ้อนของโค้ด
 - ทำให้โปรแกรมเป็นระเบียบ อ่านง่าย และดูแลรักษาง่าย
- องค์ประกอบของฟังก์ชัน
 - def ใช้ประกาศฟังก์ชัน, parameters คือค่าที่รับเข้า (Input)
 - return ใช้ส่งค่ากลับ (Output)
- รูปแบบพารามิเตอร์
 - Required arguments, Keyword arguments, Default arguments, Variable-length arguments (*args)

บทวนท้ายบท

- Lambda Function
 - ฟังก์ชันแบบสั้น ไม่ต้องตั้งชื่อ
- Recursive Function
 - ฟังก์ชันที่เรียกตัวเอง ต้องมี Base Case เพื่อหยุดการทำงาน
- ขอบเขตของตัวแปร
 - ตัวแปร Global ประกาศนอกฟังก์ชัน
 - ตัวแปร Local ประกาศภายในฟังก์ชัน
 - หากต้องการแก้ไข global ภายในฟังก์ชัน ต้องใช้คำสั่ง global